TRANSFORMERS & ATTENTION MECHANISMS

Complete Study Notes | Exam Ready

Deep Learning Unit — Architecture · Math · Examples · Exam Focus

UNIT OVERVIEW

1. Fundamentals of Transformers	2. Self-Attention Mechanism
3. Multi-Head Attention	4. Positional Encoding
5. Sequence Embeddings	6. Transformer Models (BERT & GPT)
7. Vision Transformers (ViT)	8 & 9. Transformers vs RNNs + Advantages

UNIT 1: FUNDAMENTALS OF TRANSFORMERS

Introduction to Sequence Modelling

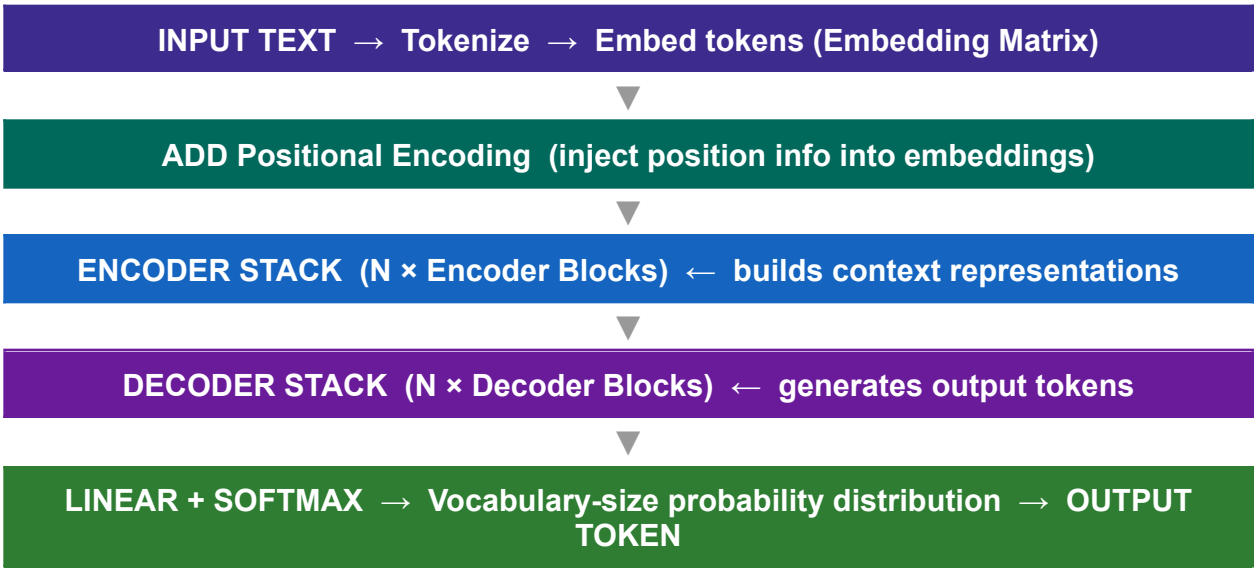
- **Sequence modelling** = processing ordered data: text, audio, time-series
- Goal: understand or generate sequences where **order matters**
- Example: 'The cat sat on the ___' — **predict the next word** using context

Limitations of RNNs and LSTMs

Problem	Why It Hurts
Sequential processing	Words processed one-by-one; cannot parallelize → very slow training
Vanishing gradients	Gradients shrink over long sequences → model forgets early context
Long-range dependencies	RNNs struggle to link word 1 to word 50 in a sentence
Fixed hidden state	All context compressed into one vector — information bottleneck

💡 KEY	Transformers were introduced in 'Attention Is All You Need' (Vaswani et al., 2017) to fix ALL of these problems simultaneously.

Transformer Architecture Overview



Transformer Components — Quick Reference

Component	Role
Input Embedding	Converts token IDs to dense vectors (d_model dimensions)

Component	Role
Positional Encoding	Adds position info; allows model to know token order
Multi-Head Attention	Lets each token attend to all others; captures relationships
Feed-Forward Network	Position-wise MLP applied to each token independently
Residual Connections	Add input to output of each sub-layer → solves vanishing gradient
Layer Normalization	Normalizes activations → stable, faster training

Encoder Block

INPUT	Token embeddings + Positional Encoding (shape: batch × seq_len × d_model)

Multi-Head Self-Attention	Add & Norm	Feed-Forward Network	Add & Norm → OUTPUT

- **Self-attention**: each token attends to ALL tokens in the same sequence
- **Add & Norm** = Residual connection + Layer Normalisation after each sub-layer
- Output shape is IDENTICAL to input shape (no dimension change)

Decoder Block

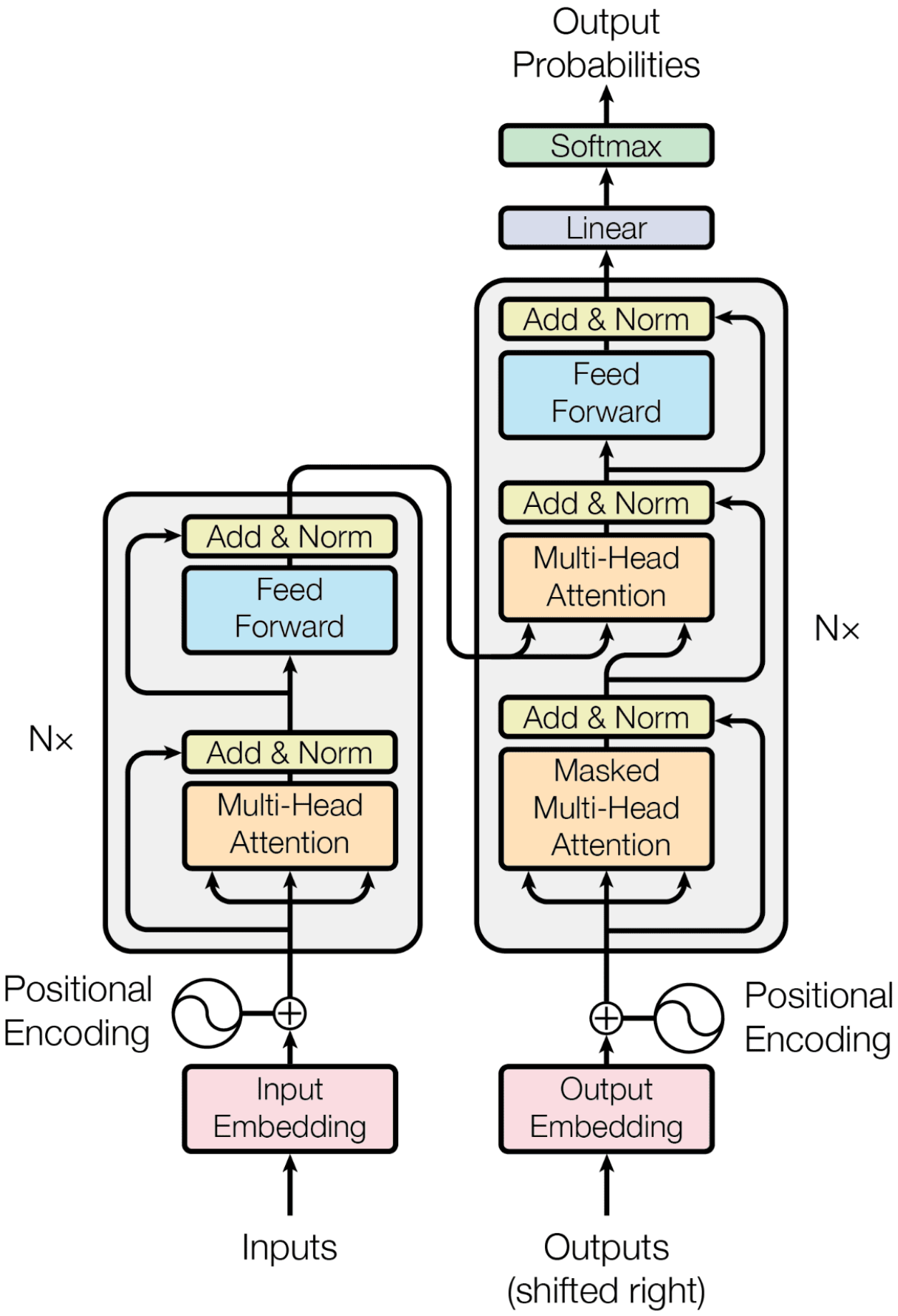
INPUT	Previously generated tokens + Encoder output (for cross-attention)

Masked Multi-Head Self-Attn	Add & Norm	Encoder-Decoder Attention	Add & Norm	Feed-Forward	Add & Norm → OUTPUT

- **Masked attention**: token i can ONLY attend to tokens 0…i (prevents future peeking)
- **Encoder-Decoder attention**: Q from decoder, K and V from encoder output

📋 Exam Focus — Unit 1

★	Encoder: only self-attention (bidirectional); used for understanding tasks
★	Decoder: masked self-attn + cross-attn (unidirectional); used for generation tasks
★	Residual connection formula: Output = LayerNorm(x + SubLayer(x))
★	Dimension flow: embedding (d_model) → attention (d_model) → FFN (4·d_model) → d_model
★	N stacked encoder/decoder blocks (typically N = 6 in original paper)


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

Step 1 of 10

Step 1: Tokenisation & token IDs

The input sentence is split into tokens. Each token maps to an integer ID from the vocabulary.

the	cat	sat	on
ID: 2	ID: 7	ID: 14	ID: 4

> Vocabulary is typically 30,000–50,000 words. Here we use a tiny vocab. Each ID is an index into the embedding table.

Step 2: Token embeddings (d=4)

Each token ID looks up a row in the embedding matrix E (vocab × d). We use d=4 for clarity. These vectors are learned during training.

EMBEDDING MATRIX LOOKUP — EACH ROW = ONE TOKEN

	e_0	e_1	e_2	e_3
the	0.62	−0.41	0.15	0.83
cat	−0.33	0.75	0.51	−0.29
sat	0.88	0.12	−0.64	0.47
on	0.19	−0.55	0.92	0.38

> These numbers are initialised randomly and refined by backprop. Similar words end up with similar vectors (cosine similarity).

- **Why Needed:** Transformers lose word order information because they process all tokens simultaneously, unlike RNNs. Positional encoding allows the model to understand that "dog bites man" is different from "man bites dog".
- **Sinusoidal Encoding:** The original Transformer uses a fixed formula with alternating sine and cosine functions of different frequencies to generate unique vectors for each position.

- **Formula:**
  - Even indices ($i = 2k$): $PE(pos, 2k) = \sin(\frac{pos}{10000^{2k/d_{model}}})$
  - Odd indices ($i = 2k + 1$): $PE(pos, 2k + 1) = \cos(\frac{pos}{10000^{2k/d_{model}}})$

- **Types of Encoding:**
  - **Absolute:** Assigns a unique code to each position in the sequence (e.g., Sinusoidal).
  - **Relative:** Encodes the distance between tokens, which can help with generalizing to unseen sequence lengths.
  - **Learned:** Instead of a fixed formula, positions are learned during training.

Why Linear (e.g., $1, 2, 3, \ldots, n$) is Inferior:
Linear encoding results in very large values for long sequences, which can create unstable gradients and fail to generalize to new lengths. YouTube · ExplainingAI +1

Step 3: Positional encoding

Transformers have no recurrence, so position must be injected. We add sinusoidal signals PE[pos, 2i] = sin(pos / 10000^(2i/d)) and PE[pos, 2i+1] = cos(…).

POSITIONAL ENCODING VECTORS

	pe_0	pe_1	pe_2	pe_3
pos 0	0.00	1.00	0.00	1.00
pos 1	0.84	0.54	0.01	1.00
pos 2	0.91	−0.42	0.02	1.00
pos 3	0.14	−0.99	0.03	1.00

AFTER ADDITION: X = EMBED + PE

	x_0	x_1	x_2	x_3
the	0.62	0.59	0.15	1.83
cat	0.51	1.29	0.52	0.71
sat	1.79	−0.30	−0.62	1.47
on	0.33	−1.54	0.95	1.38

> Adding PE means the model can tell "cat" at position 1 from "cat" at position 5 — same embedding, different positional signal.

Self attention -

Step 4: Query, Key, Value projections

Three weight matrices W^Q, W^K, W^V project each token's vector into Query, Key, Value spaces. Q asks a question; K is an index card; V is the answer.

QUERY MATRIX $Q = X \cdot W^Q$

	q_0	q_1	q_2	q_3
the	0.62	0.59	0.15	1.83
cat	0.51	1.29	0.52	0.71
sat	1.79	−0.30	−0.62	1.47
on	0.33	−1.54	0.95	1.38

KEY MATRIX $K = X \cdot W^K$

	k_0	k_1	k_2	k_3
the	0.62	0.59	0.49	1.49
cat	0.59	1.21	0.56	0.67
sat	1.58	−0.09	−0.20	1.05
on	0.14	−1.35	1.04	1.29

VALUE MATRIX $V = X \cdot W^V$

	v_0	v_1	v_2	v_3
the	0.62	0.59	0.15	1.83
cat	0.51	1.29	0.52	0.71
sat	1.79	−0.30	−0.62	1.47
on	0.33	−1.54	0.95	1.38

> In practice W^Q, W^K, W^V reduce dimensionality (e.g. d=512 → head dim 64). Here W^K is slightly off-diagonal to make scores interesting.

Step 5: Scaled dot-product attention scores

Scores = $Q \cdot K^T / \sqrt{d}$. Each row i shows how much token i wants to attend to every other token. Higher = more relevant.

$$\text{Scores} = Q \cdot K^T / \sqrt{4} \quad (\sqrt{d} = 2.0)$$

	the	cat	sat	on
the	1.77	1.20	1.41	0.91
cat	1.20	1.32	0.67	−0.11
sat	1.41	0.67	2.26	0.96
on	0.91	−0.11	0.96	2.45

> Warm cells = high score (this token pair is relevant). Dividing by $\sqrt{d}$ prevents dot products from growing large in high dimensions, keeping softmax gradients healthy.

Step 6: Softmax → attention weights

Softmax converts each row of raw scores into a probability distribution summing to 1.000.

```
Attn[i,j] = exp(score[i,j]) / Σⱼ exp(score[i,j])
```

	the	cat	sat	on
the	0.37	0.21	0.26	0.16
cat	0.34	0.38	0.20	0.09
sat	0.22	0.11	0.53	0.14
on	0.14	0.05	0.15	0.66

VISUAL: ATTENTION FROM "SAT" (ROW 2)

> Each row sums to 1.000. "sat" looks mostly at itself and nearby tokens. Warmer = higher attention weight.

Step 7: Weighted sum of Values

Attention output = Attn_weights · V. Each token's output is a weighted blend of all Value vectors.

```
AttnOut = Attn_weights · V
```

	a_0	a_1	a_2	a_3
the	0.86	0.17	0.15	1.43
cat	0.78	0.49	0.21	1.29
sat	1.18	−0.11	−0.10	1.46
on	0.60	−0.91	0.58	1.42

> Token "sat" now carries information from "the", "cat", and "on" proportional to how much it attended to them. This is context aggregation.

Multihead attention -

Input: X = embed + positional encoding

The encoder receives the sum of token embeddings and sinusoidal positional encodings. This is the same X from our earlier walkthrough — shown here as the starting matrix.

X = EMBED + PE (4 TOKENS × D=4)

	x_0	x_1	x_2	x_3
the	0.62	0.59	0.15	1.83
cat	0.51	1.29	0.52	0.71
sat	1.79	−0.30	−0.62	1.47
on	0.33	−1.54	0.95	1.38

> In a real encoder (BERT-base): d=768, sequence length up to 512, learned positional embeddings instead of sinusoidal. The matrix entering the block is shape [seq_len × d_model].

Dim = 4
Heads = 2

Multi-head attention — head 1 projections

Head 1 uses weight matrices Wq_1, Wk_1, Wv_1 each of shape [d×dh] where dh=d/H=2. It learns to attend to the first two feature dimensions.

$Q_1 = X \cdot Wq_1$, $K_1 = X \cdot Wk_1$, $V_1 = X \cdot Wv_1$ (each [4×2])

Q_1

	q_0	q_1
the	0.69	0.66
cat	0.77	1.55
sat	1.48	−0.61
on	0.80	−1.06

K_1

	k_0	k_1
the	0.62	0.59
cat	0.59	1.21
sat	1.58	−0.09
on	0.14	−1.35

V_1

	v_0	v_1
the	0.62	0.59
cat	0.51	1.29
sat	1.79	−0.30
on	0.33	−1.54

> Each head operates in a smaller subspace (dh=2 here). This lets different heads specialise — one might learn syntactic relations, another semantic ones.

Dim_head = 2

Head 1 — scaled dot-product attention

Scores = $Q_1 \cdot K_1^T / \sqrt{dh}$. Softmax converts rows to probability distributions. The output is a weighted sum of V_1.

$$scores_1 = Q_1 \cdot K_1^T / \sqrt{2}$$

RAW SCORES (HEAD 1)

	the	cat	sat	on
the	0.58	0.86	0.73	−0.57
cat	0.99	1.65	0.76	−1.41
sat	0.39	0.09	1.69	0.73
on	−0.10	−0.58	0.97	1.10

ATTENTION WEIGHTS AFTER SOFTMAX (HEAD 1)

	the	cat	sat	on
the	0.26	0.35	0.31	0.08
cat	0.26	0.51	0.21	0.02
sat	0.15	0.11	0.54	0.21
on	0.13	0.08	0.37	0.42

HEAD 1 OUTPUT = $AW_1 \cdot V_1$

	a_0	a_1
the	0.92	0.38
cat	0.80	0.71
sat	1.18	−0.25
on	0.92	−0.59

Head 2 — separate subspace

Head 2 uses its own Wq_2, Wk_2, Wv_2 and attends over a different pair of feature dimensions. Its attention pattern will differ from head 1's.

$$scores_2 = Q_2 \cdot K_2^T / \sqrt{2}$$

ATTENTION WEIGHTS (HEAD 2)

	the	cat	sat	on
the	0.37	0.13	0.20	0.30
cat	0.29	0.20	0.18	0.33
sat	0.34	0.14	0.29	0.22
on	0.33	0.15	0.13	0.39

HEAD 2 OUTPUT = $AW_2 \cdot V_2$

	b_0	b_1
the	0.29	1.48
cat	0.34	1.40
sat	0.15	1.47
on	0.41	1.44

> Head 1 vs head 2: notice the attention patterns differ — they're looking at different relationships in the same sentence. With 8 heads in BERT-base, you get 8 distinct attention patterns simultaneously.

Concatenate heads + output projection W_o

The two head outputs [4×2] are concatenated horizontally to form [4×4], then multiplied by W_o [4×4] to mix the heads and project back to d=4.

```
MHA = concat(head₁, head₂) · Wₒ
```

CONCATENATED HEADS [4×4]

	$h1_0$	$h1_1$	$h2_0$	$h2_1$
the	0.92	0.38	0.29	1.48
cat	0.80	0.71	0.34	1.40
sat	1.18	−0.25	0.15	1.47
on	0.92	−0.59	0.41	1.44

FINAL MHA OUTPUT = CONCAT · W_o [4×4]

	m_0	m_1	m_2	m_3
the	0.92	0.38	0.29	1.48
cat	0.80	0.71	0.34	1.40
sat	1.18	−0.25	0.15	1.47
on	0.92	−0.59	0.41	1.44

> W_o is learned. It blends information from all heads into a unified representation. In practice, this is the "projection layer" that follows multi-head attention.

First Add & layer norm

We add the MHA output back to the original input X (residual connection), then apply layer normalisation per token. This is the output of the first sublayer: norm1.

```
res1 = X + MHA(X) → norm1 = LayerNorm(res1)
```

AFTER RESIDUAL ADDITION (RES1)

	r_0	r_1	r_2	r_3
the	1.54	0.97	0.44	3.31
cat	1.31	2.00	0.86	2.10
sat	2.97	−0.55	−0.47	2.94
on	1.25	−2.13	1.36	2.82

AFTER LAYER NORM (NORM1) — MEAN≈0, STD≈1 PER ROW

	n_0	n_1	n_2	n_3
the	−0.03	−0.55	−1.04	1.62
cat	−0.51	0.84	−1.39	1.05
sat	1.01	−1.02	−0.98	0.99
on	0.23	−1.63	0.30	1.10

NORMALISATION EFFECT

token	μ before	σ before	μ after	σ after
the	1.565	1.079	0	1
cat	1.569	0.509	0	1
sat	1.222	1.732	0	1
on	0.827	1.813	0	1

Feed-forward network (FFN)

The FFN applies two linear transformations with a ReLU in between, position-wise (independently for each token). It expands to 4d_ff=4 (real models use 4×d, so 3072 in BERT-base) then projects back.

```
FFN(x) = max(0, x·W₁) · W₂ (ReLU activation)
```

AFTER FIRST LINEAR + RELU (NEGATIVE VALUES ZEROED)

	f_0	f_1	f_2	f_3
the	0.63	0.00	0.00	1.82
cat	0.42	0.00	0.00	2.00
sat	0.89	0.00	0.73	0.87
on	0.22	0.00	1.21	0.30

AFTER SECOND LINEAR (FFN OUTPUT)

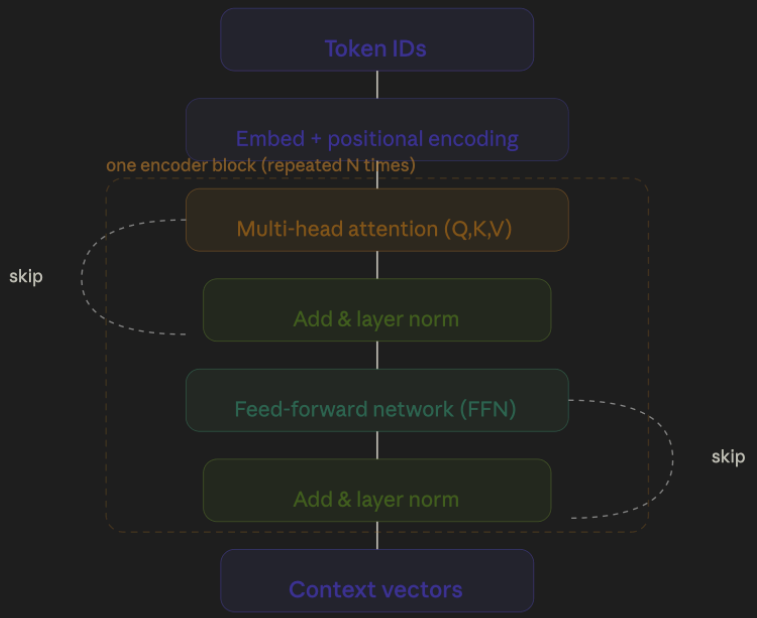
	g_0	g_1	g_2	g_3
the	0.80	0.24	−0.91	1.40
cat	0.77	0.24	−0.93	1.41
sat	0.26	0.61	−0.47	0.75
on	−0.43	0.78	0.05	−0.07

> ReLU introduces non-linearity. GELU is used in BERT/GPT instead — smoother, slightly better performance. The FFN is where most model parameters live (~2/3 of a transformer's weights).

Encoder in transformer -

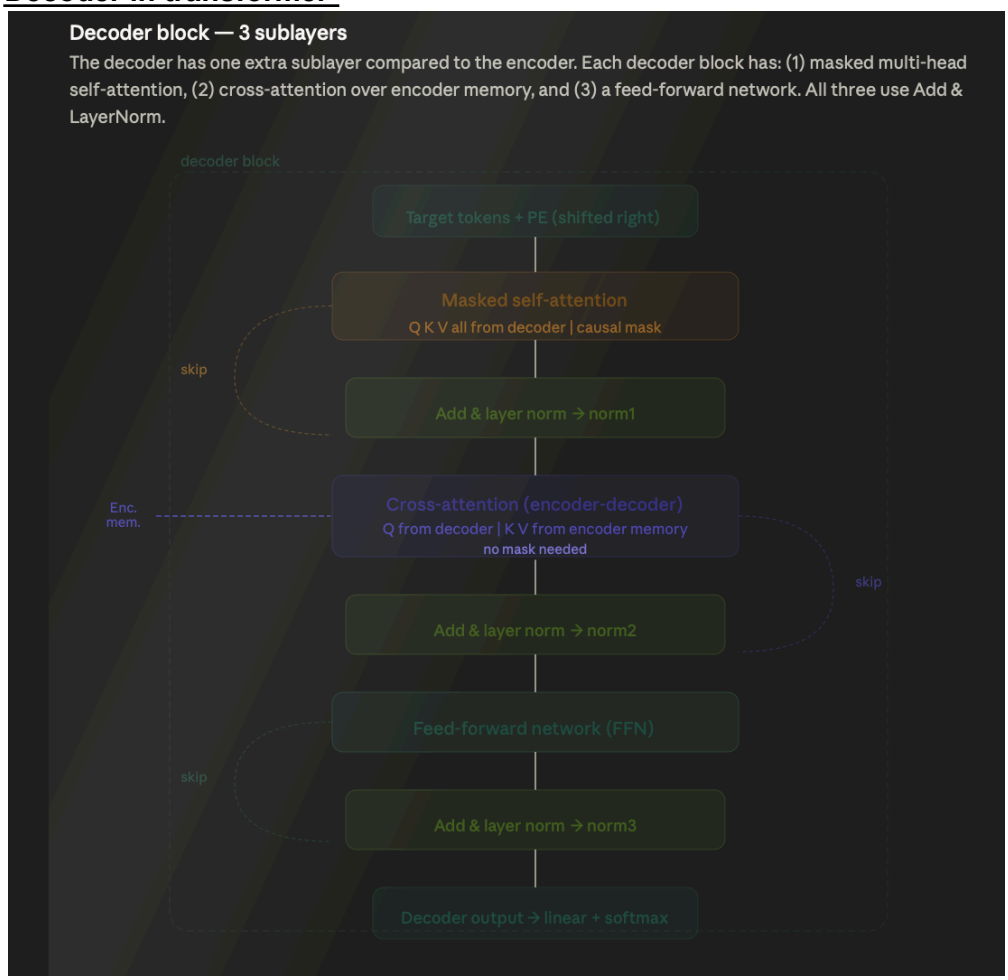
Step 10: Putting it all together

One transformer encoder block = Embed + PE → [Attention → Residual → LayerNorm] → [FFN → Residual → LayerNorm]. The decoder additionally has cross-attention over the encoder output.



> The decoder adds a third sub-layer: cross-attention over the encoder output (encoder-decoder attention). Decoder self-attention is masked so token i only attends to positions ≤ i.

Decoder in transformer-



What enters the decoder

Translation task: encoder sees French 'le chat est assis', decoder generates English token by token. At each step t, the decoder receives all previously generated tokens (shifted right), plus a special start token.

> Encoder memory (from French input — fixed for all decoder steps)

ENCODER OUTPUT — K AND V COME FROM THIS

	m_0	m_1	m_2	m_3
le	0.71	0.32	−0.48	0.85
chat	−0.22	0.91	0.63	−0.15
est	0.55	−0.38	0.77	0.42
assis	0.18	0.66	−0.29	0.93

DECODER INPUT — GENERATING STEP 3 (PRODUCING "SAT")

	d_0	d_1	d_2	d_3
the	0.45	0.68	−0.32	0.77
cat	−0.18	0.83	0.41	−0.55
sat	0.73	−0.24	0.58	0.36

> At inference time the decoder runs autoregressively — it sees ["<sos>"] to predict "the", then ["<sos>","the"] to predict "cat", etc. At training time all target tokens are fed at once with the causal mask simulating this.

Three types of attention in the decoder

Each decoder block runs three attention operations. The Q K V sources are what makes each type unique.

Sublayer 1 — Masked multi-head self-attention

Q from decoder tokens · **K** from decoder tokens · **V** from decoder tokens
Causal mask: token at position i can only attend to positions ⩽ i. Future tokens are hidden.
Purpose: lets each output token gather context from already-generated tokens.

Sublayer 2 — Cross-attention (encoder-decoder)

Q from decoder (after sublayer 1 norm) · **K** from encoder memory · **V** from encoder memory
No mask needed — decoder tokens attend freely to all source positions.
Purpose: lets the decoder "read" the full encoded source at every generation step.

Sublayer 3 — Feed-forward network

No Q/K/V — position-wise MLP: Linear → ReLU → Linear.
Same structure as encoder FFN. Applied independently per token.
Purpose: non-linear transformation to boost representational capacity.

Q from decoder
Self-attn: current decoder state
Cross-attn: norm1 output

K from decoder / encoder
Self-attn: decoder tokens
Cross-attn: encoder memory

V from decoder / encoder
Self-attn: decoder tokens
Cross-attn: encoder memory

Sublayer 1 — masked self-attention

Q K V all come from the decoder input. Raw scores are computed as $Q \cdot K^T / \sqrt{dh}$, then future positions are set to $-\infty$ before softmax, making them vanish to 0 weight.

```
scores = Q·Kᵀ / √2 then mask upper triangle to −∞
```

RAW ATTENTION SCORES

	the	cat	sat
the	0.47	0.33	0.13
cat	0.33	0.44	−0.16
sat	0.13	−0.16	0.35

AFTER CAUSAL MASK (RED = MASKED OUT = −∞ → 0 AFTER SOFTMAX)

	the	cat	sat
the	0.47	−999.00	−999.00
cat	0.33	0.44	−999.00
sat	0.13	−0.16	0.35

ATTENTION WEIGHTS AFTER SOFTMAX (MASKED POSITIONS → 0)

	the	cat	sat
the	1.00	0.00	0.00
cat	0.47	0.53	0.00
sat	0.33	0.25	0.42

> The mask enforces causality: when predicting token t, the model cannot "see" tokens t+1, t+2, … This is what makes teacher-forcing at training time equivalent to autoregressive inference.

UNIT 2: SELF-ATTENTION MECHANISM

Key Idea

IDEA	Self-attention lets every word look at every other word to decide how much to 'focus' on it when building its representation. e.g. In 'The animal didn't cross because it was tired', 'it' should attend strongly to 'animal'.

Query, Key, Value Framework

Component	Analogy	Purpose
Query (Q)	Search query in a search engine	'What am I looking for?' — represents the current token's search
Key (K)	Document metadata/tags	'What do I contain?' — represents what each token offers
Value (V)	Document content	'What information do I hold?' — actual content to be aggregated

Step-by-Step Attention Calculation

STEP 1 — Compute Q, K, V: $Q = X \cdot W_q \quad K = X \cdot W_k \quad V = X \cdot W_v$

▼

STEP 2 — Dot-Product Scores: $\text{Score}(i,j) = Q_i \cdot K^T$ (how much token i attends to token j)

▼

STEP 3 — Scale by $\sqrt{d_k}$: $\text{Scaled Score} = \text{Score} / \sqrt{d_k}$ (prevents softmax saturation)

▼

STEP 4 — Softmax: Attention Weight $\alpha = \text{softmax}(\text{Scaled Score})$ [each row sums to 1]

▼

STEP 5 — Weighted Sum: $\text{Output} = \alpha \cdot V$ (blend values by attention weights)

Attention Formula

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \cdot K^T / \sqrt{d_k} \right) \cdot V$$

Term	Meaning
Q (Queries)	Matrix of query vectors; shape (seq_len × d_k)
K (Keys)	Matrix of key vectors; shape (seq_len × d_k)

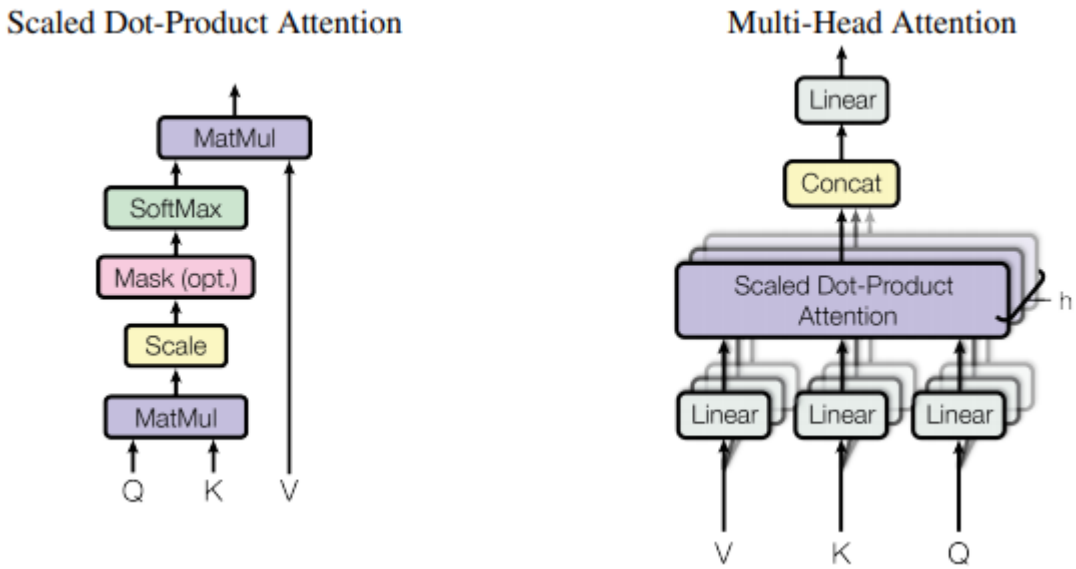
Term	Meaning
V (Values)	Matrix of value vectors; shape (seq_len × d_v)
d_k	Dimension of keys; used as scaling factor to prevent huge dot products
√d_k (scaling)	Prevents softmax from entering saturation zone → stable gradients
softmax output	Attention weights; each row is a probability distribution (sums to 1)
Output shape	(seq_len × d_v) — same length as input sequence

Worked Mini-Example

EG	Sentence: [I, love, math]. d_k=2. $Q \cdot K^T$ gives a 3×3 score matrix. After dividing by √2≈1.41 and applying softmax row-wise, we get 3×3 attention weights. Multiply by V (3×2) to get final output (3×2).

📋 Exam Focus — Unit 2

★	Formula: Attention(Q,K,V) = softmax(Q·K^T/√d_k)·V — must reproduce from memory
★	Why scale by √d_k? Large dot products → softmax saturates → vanishing gradient
★	Output shape of attention: (seq_len × d_v)
★	Score matrix shape = (seq_len × seq_len) — each pair of tokens has a score
★	Softmax converts scores to weights; each row sums to 1.0



Self-Attention

- **Mechanism:** Query (Q), Key (K), and Value (V) are derived from the same input sequence.
- **Purpose:** Allows each token to attend to all other tokens in the same sequence to understand context (e.g., resolving "it" in "the animal... it...").
- **Usage:** Encoder layers in Transformers. Medium +4

Cross-Attention

- **Mechanism:** Queries (Q) come from the previous decoder layer, while keys (K) and values (V) come from the encoder's output.
- **Purpose:** Allows the decoder to focus on specific, relevant parts of the input sequence during generation (e.g., in translation).
- **Usage:** Encoder-Decoder Attention in Transformer models. Ahead of AI | Sebastian Raschka, PhD |... +4

Multi-Head Attention

- **Mechanism:** Performs the scaled dot-product attention process multiple times in parallel, using different linear projections for Q, K, and V.
- **Purpose:** Captures different types of relationships (e.g., syntactic vs. semantic) simultaneously. The outputs are concatenated and linearly projected.
- **Usage:** Core component in modern Transformers. Ahead of AI | Sebastian Raschka, PhD |... +4

Feature	Self-Attention	Cross-Attention	Multi-Head
Source of Q, K, V	All from one sequence	Q from decoder; K, V from encoder	Multiple parallel sets
Key Goal	Intra-sequence context	Inter-sequence mapping	Capturing diverse relationships
Common Use	Transformers (Encoder)	Transformers (Decoder)	Both of the above

UNIT 3: MULTI-HEAD ATTENTION

Why Multiple Heads?

IDEA	Instead of one set of Q, K, V, run h parallel attention heads. Each head can learn a DIFFERENT type of relationship (e.g. head 1 → syntactic, head 2 → semantic, head 3 → coreference).

Multi-Head Attention Process

Input X (seq_len × d_model)

▼

LINEAR PROJECTIONS: For each head i → $Q_i = X \cdot W_i^Q$, $K_i = X \cdot W_i^K$, $V_i = X \cdot W_i^V$ [each d_model → d_k]

▼

PARALLEL ATTENTION: $head_i = Attention(Q_i, K_i, V_i)$ [run h times simultaneously]

▼

CONCATENATE: $MultiHead = Concat(head_1, head_2, \ldots, head_h)$ [shape: seq_len × (h·d_v)]

▼

FINAL LINEAR: Output = Concat · W^O [project back to d_model]

$$MultiHead(Q,K,V) = Concat(head_1,\ldots,head_h) \cdot W^O \quad \text{where } head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

Dimension Arithmetic

EXAMPLE	d_model = 512, h = 8 heads. Each head uses d_k = d_v = 512/8 = 64. After attention: each head outputs (seq_len × 64). Concat: (seq_len × 512). W_O projects (512 × 512) → output (seq_len × 512). Dimension preserved!

Symbol	Typical Value	Meaning
d_model	512	Total model dimension
h (heads)	8	Number of parallel attention heads
d_k = d_v	64 (= d_model / h)	Per-head key/value dimension
W_O shape	(h·d_v) × d_model = 512×512	Output projection matrix

Advantages

- Captures **multiple types of relationships** simultaneously (syntactic + semantic)
- Each head **specialises independently** → richer representations
- All heads computed in **parallel** → computationally efficient

📋 Exam Focus — Unit 3

★	$d_k = d_{model} / h$ (key dimension per head). Example: $d_{model}=512$, $h=8$ → $d_k=64$
★	After concat: shape = (seq_len × d_{model}) — same as input to multi-head attention
★	W_O is a learnable projection: shape ($h \cdot d_v \times d_{model}$)
★	Each head uses its OWN weight matrices W_iq, W_ik, W_iv (not shared)
★	Number of parameters in MHA: $4 \times d_{model}^2$ (Wq, Wk, Wv, Wo each $d_{model} \times d_{model}$)

UNIT 4: POSITIONAL ENCODING

The Problem

PROBLEM Self-attention is permutation-invariant — 'cat sat mat' and 'mat sat cat' produce identical attention patterns! We must inject position information.

Sinusoidal Positional Encoding

$$PE(pos, 2i) = \sin\left(pos / 10000^{(2i/d\_model)} \right)$$

$$PE(pos, 2i+1) = \cos\left(pos / 10000^{(2i/d\_model)} \right)$$

Term	Meaning
pos	Position of the token in the sequence (0, 1, 2, …, seq_len-1)
i	Dimension index (0, 1, 2, …, d_model/2 - 1)
d_model	Model embedding size (e.g. 512)
sin for even dims	Generates sine wave — smooth, periodic signal
cos for odd dims	Generates cosine wave — phase-shifted by π/2
10000^(2i/d_model)	Controls wavelength; low i = high frequency, high i = low frequency

EXAMPLE pos=0 (first token), i=0 (first dim): PE = sin(0/1) = 0.0 for even, cos(0/1) = 1.0 for odd dim.pos=1, i=0: PE = sin(1) ≈ 0.841, cos(1) ≈ 0.540. Each token gets a unique pattern!

Input Representation = Embedding + Positional Encoding

Token Embedding	+	Positional Encoding	= Final Input to Encoder

- Both have the **same shape**: (seq_len × d_model) — can be added element-wise
- **No learnable parameters** in sinusoidal PE (fully deterministic)

Types of Positional Encoding

Type	Description
Sinusoidal (original)	Fixed formula — no training needed; generalises to unseen sequence lengths
Learned PE	Position embeddings are learnable parameters (used in BERT, GPT)

Type	Description
Relative PE	Encodes relative distances between tokens; used in models like T5
Rotary PE (RoPE)	Encodes positions via rotation in complex plane; used in LLaMA/GPT-NeoX

📋 Exam Focus — Unit 4

★	Sinusoidal PE formula — know sin for even dims, cos for odd dims
★	Why sinusoidal? Can represent any position; model can learn relative offsets since sin(a+b) is linear in sin(a),cos(a)
★	PE is ADDED to embeddings (not concatenated); final shape stays d_model
★	Learned PE is used in BERT; Sinusoidal PE is from original 'Attention Is All You Need'
★	Range of PE values: between -1 and +1 (sine/cosine bounds)

UNIT 5: SEQUENCE EMBEDDINGS

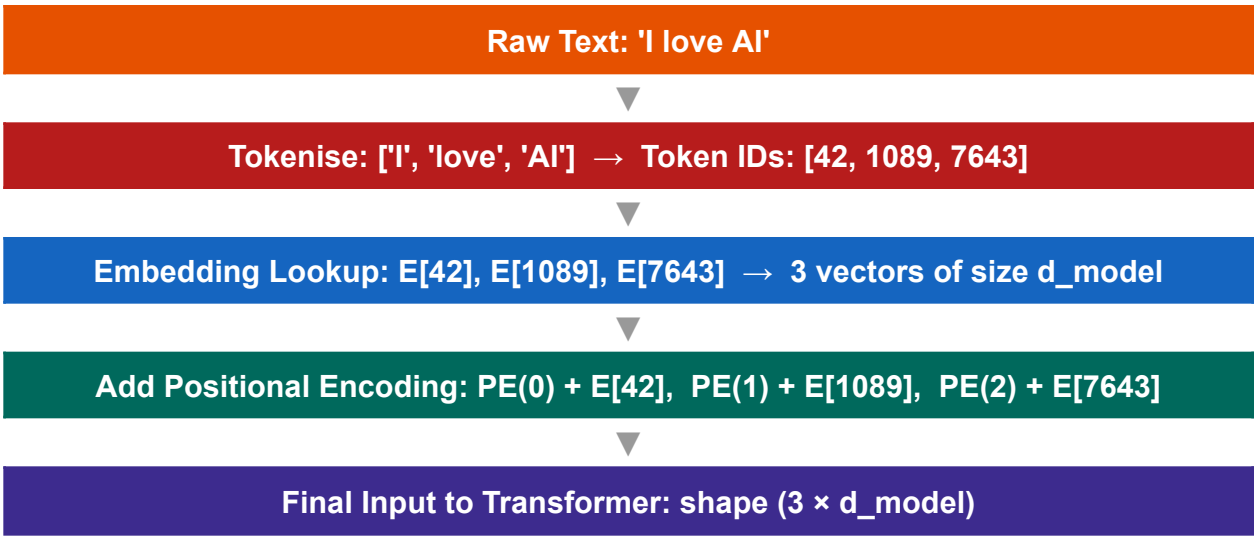
Tokenisation

- Text cannot be fed directly — must be converted to numbers
- Tokeniser splits text into tokens: words, subwords, or characters
- Example: 'unhappiness' → ['un', '##happi', '##ness'] (WordPiece/BPE)
- Each token is assigned an integer ID from a vocabulary (e.g. vocab_size = 30,000)

Embedding Matrix

DEFINITION	An Embedding Matrix E is a learnable lookup table of shape (vocab_size × d_model). Each token ID is mapped to a d_model-dimensional dense vector.

```
📐 Embedding Matrix E  shape: vocab_size × d_model   e.g. 30,000 × 512 = 15.36M parameters
```



Word Embeddings — Conceptual

Concept	Details
Word2Vec	Shallow NN; learns embeddings via skip-gram or CBOW objectives
GloVe	Global log-bilinear model; trains on word co-occurrence statistics
Contextual (BERT)	Each token gets a DIFFERENT embedding depending on context
Key property	'king' - 'man' + 'woman' ≈ 'queen' — semantics encoded in geometry

📋 Exam Focus — Unit 5

★	Embedding matrix shape: vocab_size × d_model (rows = tokens, cols = dimensions)
★	Parameters in embedding layer: vocab_size × d_model (e.g. 30000 × 512 = 15.36M)
★	Input to transformer = token embedding + positional encoding (element-wise sum)
★	Subword tokenisation (BPE/WordPiece) handles unknown words and reduces vocab size
★	Contextual embeddings (BERT) differ from static embeddings (Word2Vec/GloVe)

UNIT 6: TRANSFORMER MODELS IN NLP — BERT & GPT

BERT — Bidirectional Encoder Representations from Transformers

ARCHITECTURE	Encoder-only. Reads the ENTIRE sequence at once (bidirectional). Trained with Masked Language Modelling (MLM) and Next Sentence Prediction (NSP).

Property	BERT Details
Architecture	Encoder-only (12 or 24 stacked encoder blocks)
Directionality	Bidirectional — token can attend to tokens on BOTH left and right
Pre-training task 1	Masked LM (MLM): 15% of tokens masked; model predicts them
Pre-training task 2	Next Sentence Prediction (NSP): predict if sentence B follows A
Input special tokens	[CLS] at start; [SEP] between sentences
Output	Contextual embeddings for each token; [CLS] used for classification
Tasks	Text classification, NER, QA (extractive), sentiment analysis

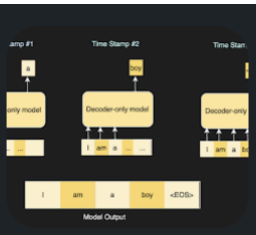
EXAMPLE	BERT fine-tuned for QA: given 'What year did WWII end? [SEP] WWII ended in 1945.', BERT extracts the span '1945' as the answer.

- **Boosting NLP Performance:** It provides a robust, pre-trained base for various NLP tasks, including:
  - **Named Entity Recognition (NER):** Identifying names, organizations, or locations.
  - **Sentiment Analysis:** Determining if text is positive or negative.
  - **Question Answering:** Finding answers in a document.
  - **Text Classification:** Categorizing text.

GPT — Generative Pre-trained Transformer

ARCHITECTURE	Decoder-only. Processes sequence left-to-right (causal/unidirectional). Trained with next-token prediction (autoregressive language modelling).

Autoregressive language modeling is **a technique where AI models generate text by predicting the next word or token in a sequence based solely on the preceding words**. It creates coherent text one step at a time, where each new token is fed back into the model as input for the next prediction. GeeksforGeeks +3



Key Characteristics

- **Sequential Generation:** Words are generated one after another, generally from left-to-right (e.g., "The" -> "sun" -> "is" -> "shining").
- **Conditional Probability:** The model calculates the likelihood of a word appearing based on the context of all previous words.
- **Dependency:** Each prediction depends on the historical context of the text already generated. GeeksforGeeks +2

Property	GPT Details
Architecture	Decoder-only (uses masked self-attention; NO encoder-decoder cross-attn)
Directionality	Unidirectional (causal) — token i can only attend to tokens ≤ i
Pre-training task	Next-token prediction: predict token t given tokens 0…t-1
Input	Prompt text; model autoregressively generates continuation
Output	Probability distribution over vocabulary at each step; sample next token
Tasks	Text generation, chatbots, code generation, story writing
Key versions	GPT-2 (1.5B params) → GPT-3 (175B) → GPT-4 (multimodal)

BERT vs GPT — Head-to-Head

Feature	BERT	GPT
Architecture	Encoder-only	Decoder-only
Attention type	Bidirectional (full attention)	Causal (masked, left-to-right)
Training task	Masked LM + NSP	Next-token prediction
Primary strength	Understanding / extracting	Generating / completing
Typical use	Classification, QA, NER	Chatbots, text generation
Context window	Fixed (512 tokens original)	Fixed but larger (1024–128k+)
Famous versions	BERT-base, RoBERTa, DistilBERT	GPT-2, GPT-3, GPT-4, ChatGPT

📋 Exam Focus — Unit 6

★	BERT = Encoder-only; Bidirectional; pre-trained with MLM & NSP
★	GPT = Decoder-only; Unidirectional (causal); pre-trained with next-token prediction
★	BERT: understanding tasks (classification, QA). GPT: generation tasks (text, code, chat)
★	[CLS] token in BERT captures sentence-level representation for classification
★	GPT uses masked attention same as Transformer decoder — no separate encoder

UNIT 7: VISION TRANSFORMERS (ViT)

Core Idea

INSIGHT	Apply the Transformer architecture to images by treating image patches as tokens — just like words in a sentence. 'An image is worth 16×16 words' (Dosovitskiy et al., 2020).

ViT Processing Pipeline

INPUT IMAGE e.g. 224×224×3 (RGB)

▼

SPLIT into patches e.g. 16×16 pixels each → Number of patches = (224/16)² = 196 patches

▼

FLATTEN each patch 16×16×3 = 768 dimensions per patch

▼

LINEAR PROJECTION 768 → d_model (learnable embedding per patch)

▼

PREPEND [CLS] token → total tokens = 197

▼

ADD Positional Encoding (1D learned; one per patch position)

▼

STANDARD TRANSFORMER ENCODER (12 blocks, no decoder)

▼

[CLS] token output → MLP head → CLASS PREDICTION

Patch Dimension Math

```
Num_patches = (H / P) × (W / P)    where H=height, W=width, P=patch_size
Patch token dim = P × P × C          where C = channels (3 for RGB)
```

EXAMPLE	Image: 224×224×3, Patch: 16×16• Num_patches = (224/16)×(224/16) = 14×14 = 196• Each patch flattened: 16×16×3 = 768 dims• After linear projection: 196 tokens of size d_model=768• Add [CLS]: 197 tokens total

ViT vs CNN

Property	ViT vs CNN
Inductive bias	ViT: minimal (no conv bias); CNN: strong (translation invariance)
Data needs	ViT needs MORE data (large pre-training); CNN works on smaller data
Global context	ViT: full global attention from layer 1; CNN: builds gradually
Scalability	ViT scales extremely well with data and model size
State of the art	ViT and hybrid models now dominate ImageNet benchmarks

Key Differences and Comparisons

- **Approach:** CNNs process images using sliding window convolutions, focusing on local, contiguous pixels. ViTs divide images into patches and apply self-attention, capturing global relationships across the entire image.
- **Data Requirements:** CNNs possess high inductive bias (assuming local, spatial relationships), enabling them to perform well on small datasets. ViTs lack this, requiring massive pre-training data to learn these spatial relationships effectively.
- **Performance:** While CNNs are reliable, ViTs generally outperform them on large-scale tasks due to higher capacity.
- **Computational Efficiency:** CNNs are generally more efficient for inference and smaller models. However, ViTs can be faster on modern GPUs due to better parallelization.
- **Memory Footprint:** During training, CNNs often have a higher memory footprint due to storing intermediate activation maps, whereas ViT tokenization reduces this load. arXiv +6

When to Choose Which?

- **Choose CNNs:** For smaller datasets, limited compute, or when working on edge devices/mobile where inference speed is crucial.
- **Choose ViTs:** When you have massive training data and high-performance computing resources, seeking the absolute best accuracy on complex tasks. Medium +2

📋 Exam Focus — Unit 7

★	Num patches = (H/P) × (W/P). Example: 224×224 image, 16×16 patch → 196 patches
★	Patch token dimension = P × P × C. Example: 16×16×3 = 768
★	ViT adds a learnable [CLS] token (like BERT) → total tokens = num_patches + 1
★	Positional encodings are LEARNED in ViT (1D, one per position)
★	ViT uses encoder-only architecture; no decoder; classification via [CLS] output + MLP

UNIT 8: TRANSFORMERS vs RNNs

Detailed Comparison

Property	RNN / LSTM	Transformer
Processing order	Sequential (token by token)	Parallel (all tokens at once)
Training speed	Slow (sequential bottleneck)	Fast (GPU-parallelisable)
Long-range dependency	Poor (vanishing gradient)	Excellent (direct attention)
Memory of context	Hidden state (fixed-size vector)	Attention over all past tokens
Gradient flow	Vanishes over long sequences	Direct paths via attention
Interpretability	Hard to inspect	Attention weights are visualisable
Data efficiency	Better on small data	Needs large data to shine
Complexity	O(n) time, O(1) memory/step	$O(n^2)$ time, $O(n^2)$ memory (attention)
Typical use today	Legacy; small sequence tasks	NLP, CV, Speech, Multimodal

Visual: Information Flow

RNN	$Word_1 \rightarrow h_1 \rightarrow Word_2 \rightarrow h_2 \rightarrow Word_3 \rightarrow h_3 \ldots \rightarrow$ Output (information flows through hidden states one step at a time; early tokens fade)
Transformer	Every word attends DIRECTLY to every other word in parallel. $Word_5$ can attend to $Word_1$ with the same strength as $Word_4$. No decay of information over distance.

📋 Exam Focus — Unit 8

★	RNN time complexity: O(n) — sequential steps. Transformer attention: $O(n^2)$ — pairwise
★	Transformers are faster to train because of parallelisation (no sequential dependency)
★	Vanishing gradient in RNNs → information lost. Transformers: residual connections fix this
★	RNNs are still used for streaming/real-time tasks (lower memory per step)
★	For sequence length n: RNN needs n steps, Transformer needs 1 step (parallel)

UNIT 9: ADVANTAGES OF ATTENTION OVER RECURRENCE

Advantages of Attention

Advantage	Explanation
Global dependencies	Any token attends to any other in a single layer — no 'telephone chain' of hidden states
Parallel computation	All attention computations are independent → exploits modern GPU parallelism
Better gradient flow	Attention provides direct gradient paths between any two tokens → no vanishing
Interpretability	Attention weights are human-readable: 'which tokens did this token look at?'
Scalability	'Scaling laws' show transformers keep improving with more data and parameters
Transfer learning	Large pre-trained transformers (BERT, GPT) fine-tune extremely well on small tasks

Limitations of Attention

Limitation	Detail
Quadratic memory $O(n^2)$	Attention matrix is seq_len × seq_len; for n=10k this is 100M entries
Quadratic compute $O(n^2)$	All pairs of tokens must interact — expensive for very long sequences
Requires large data	Without enough pre-training data, transformers underfit vs RNNs
Positional encoding hack	Order info is 'bolted on'; not intrinsic to the architecture
No state between calls	Transformers are stateless at inference; must re-process context each time

Solutions to Limitations

- **Sparse attention** (Longformer, BigBird): reduce $O(n^2)$ to $O(n)$ by attending only to local + global tokens
- **Linear attention** (Performers): approximate full attention in $O(n)$ time
- **Flash Attention**: IO-aware exact attention; drastically reduces memory usage
- **State Space Models** (**Mamba**): combines RNN efficiency with attention-like global context

📋 Exam Focus — Unit 9

★ Attention complexity: $O(n^2)$ in both time and memory — key limitation for long sequences

★	Three key advantages: (1) global dependencies, (2) parallelism, (3) better gradients
★	Sparse attention (Longformer): attends to local window + [CLS] → O(n) complexity
★	Scalability: transformers exhibit 'scaling laws' — performance improves predictably with scale
★	Attention vs recurrence: attention wins on quality; RNN wins on memory per-step efficiency

MASTER CHEAT SHEET — KEY FORMULAE & FACTS

Formula / Fact	Detail
Attention formula	$\text{Attention}(Q,K,V) = \text{softmax}(QK^T/\sqrt{d_k})\cdot V$
Multi-head formula	$\text{MultiHead} = \text{Concat}(\text{head}_1 \ldots \text{head}_h)\cdot W_O$
d_k per head	$d_k = d_{model} / h$ (e.g. 512/8 = 64)
Positional Encoding (even)	$PE(pos,2i) = \sin(pos / 10000^{(2i/d_{model})})$
Positional Encoding (odd)	$PE(pos,2i+1) = \cos(pos / 10000^{(2i/d_{model})})$
Residual connection	$\text{Output} = \text{LayerNorm}(x + \text{SubLayer}(x))$
ViT num patches	$(H/P) \times (W/P)$ e.g. $(224/16)^2 = 196$
ViT patch dim	$P \times P \times C$ e.g. $16\times16\times3 = 768$
Embedding params	$\text{vocab\_size} \times d_{model}$ e.g. 30k × 512 = 15.36M
Attention complexity	$O(n^2 \cdot d)$ time; $O(n^2)$ memory (n=seq_len)
BERT	Encoder-only · Bidirectional · MLM+NSP pre-training
GPT	Decoder-only · Causal · Next-token prediction
ViT	Encoder-only · Patch-as-token · 1D learned PE

Good luck on your exam! ⚡ Attention is all you need!